Script oral — soutenance de stage (CIN · Avatar-ID)

Durée visée : **environ 9 min**, démonstration de 40 s comprise. Rythme : environ 130 mots/min. `[clic]` = cliquer pour faire apparaître l'élément suivant.

#	Diapositive	Durée	Cumul
1	Détection automatique et autocapture de la CIN tunisienne	0:20	0:20
2	Mon module est la première étape du parcours Avatar-ID : sa qualité conditionne la suite	0:30	0:50
3	Une photo libre de CIN peut être floue, de travers ou envoyée deux fois	0:30	1:20
4	Cinq fonctions à automatiser, trois cibles chiffrées, deux plateformes	0:35	1:55
5	Seul OpenCV tenait Android + iOS en six semaines, sans jeu de données annoté	0:35	2:30
6	Un domaine sans Flutter ni OpenCV rend la logique testable sans caméra	0:30	3:00
7	Quatre passes de contours et un score géométrique retiennent le quadrilatère de la carte	1:00	4:00
8	Sur fond clair, la détection échouait ; médiane, CLAHE et réglage des seuils l'ont corrigée	0:40	4:40
9	La capture part seule après deux frames consécutives nettes, bien exposées et stables	0:45	5:25
10	Un visage et deux gabarits identifient le recto ; trois indices votent pour le verso	0:40	6:05
11	Après les deux captures, l'utilisateur vérifie, ajuste et exporte chaque face	0:25	6:30
12	Démonstration : de la carte posée devant la caméra à la page A4	0:45	7:15
13	177 cas de test automatisés passent sans échec, organisés en quatre paliers	0:25	7:40
14	Les trois cibles du cahier des charges sont atteintes sur appareil réel	0:30	8:10
15	Quatre pistes pour amener le module vers la production	0:20	8:30
16	Une capture recto/verso autonome, sur Android et iOS, livrée en six semaines	0:35	9:05
17	Merci pour votre attention	0:05	9:10

1. Détection automatique et autocapture de la CIN tunisienne

0:20 — cumul 0:20

Bonjour. Je présente mon stage d'été de six semaines chez Codingaholic, encadré par M. Jendoubi : un module mobile qui détecte et capture automatiquement le recto puis le verso de la carte d'identité tunisienne, pour l'application Avatar-ID.

Transition : « D'abord, le contexte. »

2. Mon module est la première étape du parcours Avatar-ID : sa qualité conditionne la suite

0:30 — cumul 0:50 · clics : 2 clics : 1) étapes 2 à 5 d'Avatar-ID, après « Mon module est la première » ; 2) les deux conclusions du bas.

Avatar-ID vérifie l'identité à distance, comme un KYC bancaire. Mon module en est la première étape : la capture de la carte. **[clic]** Ensuite, Avatar-ID réalise le test de vivacité, compare la photo de la carte au visage, lit la carte par OCR et enregistre le résultat. **[clic]** La qualité de ma capture conditionne donc toute la suite.

Transition : « Or cette capture échoue souvent. »

3. Une photo libre de CIN peut être floue, de travers ou envoyée deux fois

0:30 — cumul 1:20 · clics : 4 clics : les trois défauts un par un (flou, de travers, deux fois le recto), puis la problématique.

Une simple photo de la carte est souvent **[clic]** floue, **[clic]** de travers, **[clic]** ou c'est deux fois le recto au lieu du verso. **[clic]** D'où ma problématique : détecter en temps réel et de façon fiable les contours de la CIN, quels que soient l'éclairage et l'arrière-plan.

Transition : « Voici ce que le module devait faire. »

4. Cinq fonctions à automatiser, trois cibles chiffrées, deux plateformes

0:35 — cumul 1:55 · clics : 3 clics : 1) les cinq fonctions ; 2) les trois cibles ; 3) Android et iOS.

[clic] Cinq fonctions : détecter la carte, contrôler la netteté, la luminosité et la stabilité, distinguer le recto du verso, capturer sans bouton, puis recadrer et améliorer. **[clic]** Trois cibles : moins de 100 millisecondes par image, au moins 90 % de recto/verso reconnus en conditions normales, et environ deux secondes jusqu'à la capture. **[clic]** Le tout sur Android et iOS, avec un seul code, en six semaines.

Transition : « Cette double plateforme a dicté la technologie. »

5. Seul OpenCV tenait Android + iOS en six semaines, sans jeu de données annoté

0:35 — cumul 2:30 · clics : 2 clics : 1) la pile retenue (logos) ; 2) la remarque en orange.

Le Document Scanner de ML Kit n'existe que sur Android. Un modèle entraîné sur mesure exigeait plusieurs centaines d'images de CIN annotées, impossible à constituer en six semaines. J'ai donc retenu OpenCV : multiplateforme et sans entraînement, au prix d'un réglage

empirique des seuils. **[clic]** La pile : Flutter et Dart pour l'application, OpenCV via le binding opencv_dart. **[clic]** En orange, le critère qui élimine chaque option.

Transition : « Voyons l'organisation du code. »

6. Un domaine sans Flutter ni OpenCV rend la logique testable sans caméra

0:30 — cumul 3:00 · clics : 1 clic : la matrice des sept modules, après avoir expliqué le module type.

Le code suit la Clean Architecture, découpée par fonctionnalité. La couche présentation dépend du domaine, et la couche données l'implémente. La clé est le domaine : il ne dépend ni de Flutter ni d'OpenCV, donc la machine à états ou la géométrie se testent sans caméra ni émulateur. **[clic]** Chaque module ne prend que les couches dont il a besoin.

Transition : « Passons au cœur : la détection. »

7. Quatre passes de contours et un score géométrique retiennent le quadrilatère de la carte

1:00 — cumul 4:00 · clics : 1 clic : la bande « conçu et développé pendant le stage », au moment de « Autour d'OpenCV, j'ai conçu ».

Chaque image caméra, en YUV420 ou BGRA, devient une matrice OpenCV. Un CLAHE, une égalisation d'histogramme locale, renforce le contraste. Quatre passes produisent ensuite des masques de contours : Canny, avec des seuils à 0,66 et 1,33 fois la médiane ; un seuillage adaptatif ; la saturation HSV avec Otsu ; et, en dernier recours, un gradient morphologique. Chaque contour est simplifié en polygone ; un score de 80 % sur la proportion du format ID-1 et 20 % sur la taille retient le meilleur quadrilatère au-dessus de 0,30. Une homographie le redresse en 856 par 540 pixels. **[clic]** Autour de ces primitives, j'ai conçu la sortie anticipée à 0,65, le suivi à plus ou moins 20 % autour de la position précédente, les garde-fous contre un faux quadrilatère, et un isolate Dart qui garde l'aperçu fluide.

Transition : « Cela n'a pas marché du premier coup. »

8. Sur fond clair, la détection échouait ; médiane, CLAHE et réglage des seuils l'ont corrigée

0:40 — cumul 4:40 · clics : 3 clics : 1) les deux causes ; 2) les trois correctifs ; 3) le résultat observé.

Sur fond clair, une table blanche par exemple, la détection échouait souvent. **[clic]** Deux causes : un bug, les seuils de Canny calculés sur la moyenne au lieu de la médiane, donc trop hauts sur une scène claire ; et un contraste local trop faible. **[clic]** Trois correctifs : la vraie médiane, un CLAHE avant la détection, et le réglage des seuils de score et d'aire. **[clic]** Résultat : une détection comparable sur fond clair et sur fond sombre.

Transition : « Une fois la carte trouvée, quand capturer ? »

9. La capture part seule après deux frames consécutives nettes, bien exposées et stables

0:45 — cumul 5:25 · clics : 1 clic : les cinq contrôles, après avoir décrit la machine à états.

L'autocapture est une machine à états : searching tant qu'aucune carte n'est trouvée, poorQuality tant qu'un contrôle échoue, holding pendant le maintien, puis triggerCapture et captured. Si un contrôle échoue, on revient en arrière. **[clic]** Cinq contrôles : la netteté, mesurée par la variance du Laplacien, au moins 120 ; la luminosité, avec au plus 12 % de pixels saturés pour écarter les reflets ; la stabilité, une dérive des coins sous 2,5 % de la diagonale ; la bonne face ; et un maintien de deux images consécutives. C'est un compteur d'images, pas une minuterie.

Transition : « La bonne face, justement. »

10. Un visage et deux gabarits identifient le recto ; trois indices votent pour le verso

0:40 — cumul 6:05 · clics : 3 clics : 1) le recto ; 2) le verso ; 3) l'anti-doublon.

[clic] Au recto, la photo est toujours en haut à gauche : j'y cherche un visage avec la cascade de Haar d'OpenCV, pré-entraînée, sur quatre rotations, ce qui donne aussi l'orientation. Le logo et le drapeau sont confirmés par template matching, avec une corrélation d'au moins 0,6. **[clic]** Le verso n'a pas de visage : le code-barres, la texture de l'empreinte et la ligne de séparation, trouvée par une transformée de Hough, votent sur quatre rotations. **[clic]** Ainsi, présenter deux fois le recto ne déclenche rien.

Transition : « Après la capture, l'utilisateur garde la main. »

11. Après les deux captures, l'utilisateur vérifie, ajuste et exporte chaque face

0:25 — cumul 6:30 · clics : 3 clics : une capture d'écran par clic (vérification, recadrage, page A4).

Une fois les deux faces capturées, l'utilisateur **[clic]** vérifie et règle contraste et netteté, **[clic]** ajuste le recadrage si besoin, **[clic]** puis exporte le recto, le verso et une page A4 à 300 dpi. Tout est traité sur l'appareil.

Transition : « Voyons tout le parcours en vidéo. »

12. Démonstration : de la carte posée devant la caméra à la page A4

0:45 — cumul 7:15

> **Vidéo :** 40 s environ. Lancer la vidéo dès l'arrivée sur la diapositive (clic sur la vidéo). Pour l'intégrer : déposer le fichier dans build/assets/demo.mp4 puis relancer le script, ou dans PowerPoint : Insertion → Vidéo → Cet appareil, à la place du cadre, puis Lecture → Démarrer « Automatiquement ».

Voici le module en action. **[lancer la vidéo]** Le cadre est rouge tant que la carte n'est pas détectée, puis passe au vert ; les messages guident l'utilisateur. Le recto est capturé tout seul dès

que la netteté, la luminosité et la stabilité sont bonnes, puis l'application passe au verso et le capture à son tour. On arrive ensuite sur la vérification, l'ajustement et l'export de la page A4.

Transition : « Comment l'ai-je validé ? »

13. 177 cas de test automatisés passent sans échec, organisés en quatre paliers

0:25 — cumul 7:40 · clics : 4 clics : un palier de test par clic (A, B, C, D).

177 tests automatisés passent sans échec, répartis en quatre paliers : **[clic]** la logique pure Dart, **[clic]** la plateforme simulée avec mocktail, **[clic]** OpenCV réel sur images synthétiques, **[clic]** et les vraies cartes sur téléphone, avec un protocole manuel.

Transition : « Et sur appareil réel ? »

14. Les trois cibles du cahier des charges sont atteintes sur appareil réel

0:30 — cumul 8:10 · clics : 3 clics : une mesure par clic (temps, reconnaissance, délai) ; Android et iOS arrive avec la troisième.

Les trois cibles sont atteintes : **[clic]** 92 à 100 millisecondes de traitement par image dans l'isolate, **[clic]** 92 % de recto/verso reconnus sur cartes réelles, **[clic]** et environ deux secondes entre le cadre stabilisé et la capture, avec le même comportement sur Android et iOS.

Transition : « Pour la suite. »

15. Quatre pistes pour amener le module vers la production

0:20 — cumul 8:30 · clics : 4 clics : une piste par clic.

Quatre pistes vers la production : **[clic]** un jeu d'images plus large, **[clic]** des tests réels automatisés, **[clic]** un message de timeout, **[clic]** et une analyse de texture contre les cartes affichées sur un écran.

Transition : « Pour conclure. »

16. Une capture recto/verso autonome, sur Android et iOS, livrée en six semaines

0:35 — cumul 9:05 · clics : 4 clics : 1) livré ; 2) mesuré ; 3) perspectives ; 4) mon apport.

En résumé, **[clic]** j'ai livré un module complet en six semaines, **[clic]** qui atteint ses trois cibles sur Android et iOS, **[clic]** avec des pistes claires pour la suite. **[clic]** Mon apport, c'est tout ce qui entoure les primitives d'OpenCV : l'enchaînement des passes, le score, l'isolate, la machine à états et le vote recto/verso. J'en retiens qu'un choix technique se justifie par les contraintes réelles du projet.

Transition : « Merci de votre attention. »

17. Merci pour votre attention

0:05 — cumul 9:10

Merci de votre attention ; je suis à votre disposition pour vos questions.

Conseils. Répéter deux fois avec un chronomètre. Si le temps manque, raccourcir les diapositives 5 et 6 (choix technologique, architecture) plutôt que la démonstration ou les résultats. Les questions probables et leurs réponses sont dans les notes de chaque diapositive et dans `QA_Entrainement`.